

?GROUP 8 API + POSTMAN FULL GUIDE (REQUIRED ENDPOINTS)

Date: 2026-05-24

Audience: Groupmates who need a complete, copy-paste workflow.

0) WHY YOU GET 400 BAD REQUEST IN POSTMAN (EVEN IF APP WORKS)

Your backend enforces encrypted transport payloads for API requests and responses.

What this means:

- If request has a body, backend expects header: X-Payload-Encryption: v1
- Request body must be JSON envelope: { "payload": "<base64 opaque payload>" }
- Sending plain JSON like {"email":"...","password":"..."} will fail

Backend sources:

- api/index.php wires encrypted transport for all requests/responses
- backend/src/Http/Request.php validates/decrypts incoming body
- backend/src/Http/AesGcmPayloadEnvelope.php enforces header + envelope format

1) ONE-TIME POSTMAN SETUP

1. Create an environment in Postman (example name: Group8 Local).
2. Add these environment variables:

- base_url
Value: http://localhost/group8/api
- transport_key
Value: same key material as backend APP_ENCRYPTION_KEY
(Frontend uses this as VITE_TRANSPORT_ENCRYPTION_KEY.)
- auth_token
Value: leave blank initially
- last_decrypted_json
Value: leave blank initially

3. Create one collection (example name: Group8 Required Endpoints).
4. Add all requests under this collection.
5. Paste the scripts below at collection level (so all requests inherit them):
 - Collection Script > Pre-request
 - Collection Script > Post-response

2) COLLECTION PRE-REQUEST SCRIPT (AUTO ENCRYPT BODY)

Copy-paste this exactly into Collection > Scripts > Pre-request:

```
const HEADER_NAME = 'X-Payload-Encryption';
const HEADER_VALUE = 'v1';

function decodeBase64(value) {
  const binary = atob(value);
  const bytes = new Uint8Array(binary.length);
  for (let i = 0; i < binary.length; i += 1) {
    bytes[i] = binary.charCodeAt(i);
  }
  return bytes;
}

function encodeBase64(bytes) {
```

```

    let binary = '';
    for (const b of bytes) {
        binary += String.fromCharCode(b);
    }
    return btoa(binary);
}

function decodeHex(value) {
    const out = new Uint8Array(value.length / 2);
    for (let i = 0; i < value.length; i += 2) {
        out[i / 2] = Number.parseInt(value.slice(i, i + 2), 16);
    }
    return out;
}

function resolveKeyBytes(rawKey) {
    const candidate = String(rawKey || '').trim();
    if (!candidate) {
        throw new Error('Missing transport_key variable.');
```

}

 let resolved = null;

 if (candidate.startsWith('base64:')) {
 resolved = decodeBase64(candidate.slice(7));
 } else if (/^[0-9a-fA-F]{64}\$/.test(candidate)) {
 resolved = decodeHex(candidate);
 } else if (/^[A-Za-z0-9+/*] += *\$/.test(candidate)) {
 try {
 const decoded = decodeBase64(candidate);
 if (decoded.byteLength === 32) {
 resolved = decoded;
 }
 } catch (err) {
 // continue to raw text mode
 }
 }

 if (!resolved) {
 resolved = new TextEncoder().encode(candidate);
 }

 if (resolved.byteLength !== 32) {
 throw new Error('transport_key must resolve to exactly 32 bytes.');

}

 return resolved;
}

function packOpaquePayload(iv, tag, cipherText) {
 const packed = new Uint8Array(iv.byteLength + tag.byteLength + cipherText.byteLength);
 packed.set(iv, 0);
 packed.set(tag, iv.byteLength);
 packed.set(cipherText, iv.byteLength + tag.byteLength);
 return encodeBase64(packed);
}

async function encryptPayload(bodyObject, keyBytes) {
 const key = await crypto.subtle.importKey('raw', keyBytes, { name: 'AES-GCM' }, false,
 ['encrypt']);
 const iv = crypto.getRandomValues(new Uint8Array(12));
 const plain = new TextEncoder().encode(JSON.stringify(bodyObject));

```

const encryptedBuffer = await crypto.subtle.encrypt(
  { name: 'AES-GCM', iv, tagLength: 128 },
  key,
  plain,
);

const encrypted = new Uint8Array(encryptedBuffer);
const tagLength = 16;
if (encrypted.byteLength <= tagLength) {
  throw new Error('Invalid AES-GCM output length.');
```

```

}

const cipherText = encrypted.slice(0, encrypted.byteLength - tagLength);
const tag = encrypted.slice(encrypted.byteLength - tagLength);

return { payload: packOpaquePayload(iv, tag, cipherText) };
}

(async () => {
  const transportKey =
    pm.environment.get('transport_key') ||
    pm.collectionVariables.get('transport_key') ||
    pm.variables.get('transport_key');

  const keyBytes = resolveKeyBytes(transportKey);

  // Always enforce required transport headers.
  pm.request.headers.upsert({ key: 'Content-Type', value: 'application/json' });
  pm.request.headers.upsert({ key: HEADER_NAME, value: HEADER_VALUE });

  // Auto-attach bearer token if available.
  const token = String(pm.environment.get('auth_token') || '').trim();
  if (token) {
    pm.request.headers.upsert({ key: 'Authorization', value: 'Bearer ${token}' });
  } else {
    pm.request.headers.remove('Authorization');
  }

  // Encrypt only raw JSON bodies.
  if (!pm.request.body || pm.request.body.mode !== 'raw') {
    return;
  }

  const raw = String(pm.request.body.raw || '').trim();
  if (!raw) {
    return;
  }

  const parsed = JSON.parse(raw);

  // Prevent accidental double-encryption.
  if (
    parsed &&
    typeof parsed === 'object' &&
    !Array.isArray(parsed) &&
    typeof parsed.payload === 'string' &&
    Object.keys(parsed).length === 1
  ) {
    return;
  }

  const encryptedEnvelope = await encryptPayload(parsed, keyBytes);
  pm.request.body.update(JSON.stringify(encryptedEnvelope));

```

```
})();
```

```
=====
3) COLLECTION POST-RESPONSE SCRIPT (AUTO DECRYPT + AUTO TOKEN)
=====
```

Copy-paste this exactly into Collection > Scripts > Post-response:

```
-----

const HEADER_NAME = 'X-Payload-Encryption';
const HEADER_VALUE = 'v1';

function decodeBase64(value) {
    const binary = atob(value);
    const bytes = new Uint8Array(binary.length);
    for (let i = 0; i < binary.length; i += 1) {
        bytes[i] = binary.charCodeAt(i);
    }
    return bytes;
}

function decodeHex(value) {
    const out = new Uint8Array(value.length / 2);
    for (let i = 0; i < value.length; i += 2) {
        out[i / 2] = Number.parseInt(value.slice(i, i + 2), 16);
    }
    return out;
}

function resolveKeyBytes(rawKey) {
    const candidate = String(rawKey || '').trim();
    if (!candidate) {
        throw new Error('Missing transport_key variable.');

```
 }

 let resolved = null;

 if (candidate.startsWith('base64:')) {
 resolved = decodeBase64(candidate.slice(7));
 } else if (/^[0-9a-fA-F]{64}$/.test(candidate)) {
 resolved = decodeHex(candidate);
 } else if (/^[A-Za-z0-9+\/]+=*$/.test(candidate)) {
 try {
 const decoded = decodeBase64(candidate);
 if (decoded.byteLength === 32) {
 resolved = decoded;
 }
 } catch (err) {
 // continue to raw text mode
 }
 }

 if (!resolved) {
 resolved = new TextEncoder().encode(candidate);
 }

 if (resolved.byteLength !== 32) {
 throw new Error('transport_key must resolve to exactly 32 bytes.');

```
    }

    return resolved;
}
```


```


```

```

function unpackOpaquePayload(value) {
  const packed = decodeBase64(value);
  if (packed.byteLength <= 28) {
    throw new Error('Invalid encrypted payload length.');
```

```

  }

  const iv = packed.slice(0, 12);
  const tag = packed.slice(12, 28);
  const cipherText = packed.slice(28);

  const encrypted = new Uint8Array(cipherText.byteLength + tag.byteLength);
  encrypted.set(cipherText, 0);
  encrypted.set(tag, cipherText.byteLength);

  return { iv, encrypted };
}

async function decryptEnvelope(envelope, keyBytes) {
  if (!envelope || typeof envelope !== 'object' || typeof envelope.payload !== 'string') {
    throw new Error('Response does not contain encrypted payload envelope.');
```

```

  }

  const key = await crypto.subtle.importKey('raw', keyBytes, { name: 'AES-GCM' }, false,
    ['decrypt']);
  const unpacked = unpackOpaquePayload(envelope.payload);

  const plainBuffer = await crypto.subtle.decrypt(
    { name: 'AES-GCM', iv: unpacked.iv, tagLength: 128 },
    key,
    unpacked.encrypted,
  );

  const text = new TextDecoder().decode(new Uint8Array(plainBuffer));
  return JSON.parse(text);
}

(async () => {
  const marker = pm.response.headers.get(HEADER_NAME);
  if (String(marker || '').trim() !== HEADER_VALUE) {
    // Not an encrypted payload response; do nothing.
    return;
  }

  const transportKey =
    pm.environment.get('transport_key') ||
    pm.collectionVariables.get('transport_key') ||
    pm.variables.get('transport_key');

  const keyBytes = resolveKeyBytes(transportKey);
  const envelope = pm.response.json();
  const decrypted = await decryptEnvelope(envelope, keyBytes);

  // Save decrypted JSON for easy viewing and debugging.
  pm.environment.set('last_decrypted_json', JSON.stringify(decrypted, null, 2));

  // Auto-store token from login/register responses.
  if (decrypted && typeof decrypted.token === 'string' && decrypted.token.trim() !== '') {
    pm.environment.set('auth_token', decrypted.token);
  }

  console.log('Decrypted response:', decrypted);
})();
-----

```

How to view decrypted response quickly:

- Open Postman Console, then Send a request.
- You will see: Decrypted response: { ... }
- Also check environment variable last_decrypted_json.

4) REQUIRED ENDPOINTS CHECKLIST (FROM endpoint-docs.ts)

Use this exact list for your report/demo:

[Auth]

1. POST {{base_url}}/auth/register
2. POST {{base_url}}/auth/login

[Profile and Users]

3. GET {{base_url}}/users/profile
4. PUT {{base_url}}/users/profile

[Exams]

5. POST {{base_url}}/exams
6. GET {{base_url}}/exams
7. GET {{base_url}}/exams/{{exam_id}}

[Results]

8. POST {{base_url}}/results/submit
9. GET {{base_url}}/results/student/{{student_id}}

[Admin and Reporting]

10. GET {{base_url}}/admin/exams
11. GET {{base_url}}/admin/results
12. GET {{base_url}}/reports/exam-performance
13. GET {{base_url}}/reports/pass-fail

5) REQUEST-BY-REQUEST SPOONFEED (COPY-PASTE)

Important:

- For requests WITH body, you type plain JSON below.
- Pre-request script encrypts automatically.
- For requests WITHOUT body, leave Body empty.

[1] REGISTER USER

Method: POST

URL: {{base_url}}/auth/register

Auth token needed: No

Body (raw JSON):

```
{
  "name": "Student One",
  "email": "student1@example.com",
  "password": "Pass1234!",
  "role": "student",
  "department": "Science"
}
```

Code notes:

- Public register blocks role=admin.
- Use role student or teacher only.

[2] LOGIN USER

Method: POST

URL: {{base_url}}/auth/login

Auth token needed: No

Body (raw JSON):

```
{
  "email": "student1@example.com",
  "password": "Pass1234!"
}
```

After successful login:

- Collection post-response script auto-saves auth_token.

[3] GET PROFILE

Method: GET

URL: {{base_url}}/users/profile

Auth token needed: Yes

Body: none

[4] UPDATE PROFILE

Method: PUT

URL: {{base_url}}/users/profile

Auth token needed: Yes

Body (raw JSON):

```
{
  "name": "Student One Updated",
  "department": "Engineering",
  "phone": "+1 555 1000",
  "bio": "Testing API via Postman"
}
```

Notes:

- name cannot be empty.
- If field omitted, backend keeps existing value.

[5] CREATE EXAM

Method: POST

URL: {{base_url}}/exams

Auth token needed: Yes (teacher or admin)

Body (raw JSON):

```
{
  "title": "Algebra Quiz 1",
  "description": "Basic algebra quiz",
  "classId": "c1",
  "teacherId": "u_teacher_1",
  "duration": 60,
  "totalMarks": 10,
  "passingMarks": 6,
  "startDate": "2026-05-25T08:00:00Z",
  "endDate": "2026-05-25T09:30:00Z",
  "status": "published",
  "questions": [
    {
```

```

        "id": "q1",
        "text": "2 + 2 = ?",
        "type": "mcq",
        "topic": "Arithmetic",
        "options": ["3", "4", "5"],
        "correctAnswer": "4",
        "marks": 10
    }
]
}

```

Critical validator rules:

- status must be draft, published, or completed.
- endDate must be later than startDate.
- For mcq: at least 2 options and correctAnswer must match one option.
- Sum of question marks must equal totalMarks exactly.
- If caller is admin, teacherId is required.

[6] LIST EXAMS

Method: GET
 URL: {{base_url}}/exams
 Auth token needed: Yes
 Body: none

[7] GET EXAM BY ID

Method: GET
 URL: {{base_url}}/exams/{{exam_id}}
 Auth token needed: Yes
 Body: none

How to set exam_id:

- Copy from create/list response and set env var exam_id.

[8] SUBMIT RESULT

Method: POST
 URL: {{base_url}}/results/submit
 Auth token needed: Yes (student)

Body (raw JSON):

```

{
  "examId": "{{exam_id}}",
  "answers": [
    {
      "questionId": "q1",
      "answer": "4"
    }
  ],
  "submittedAt": "2026-05-25T08:20:00Z",
  "questionTelemetry": [
    {
      "questionId": "q1",
      "timeSpentSeconds": 40,
      "visitCount": 1,
      "answerChangeCount": 0
    }
  ]
}

```

Notes:

- answers must be array.
- every answer row needs questionId.
- For most flows, exam status should be published.

[9] GET RESULTS BY STUDENT

Method: GET
URL: {{base_url}}/results/student/{{student_id}}
Auth token needed: Yes
Body: none

How to set student_id:
- Use logged in student user id from decrypted login response.

[10] ADMIN EXAMS DASHBOARD

Method: GET
URL: {{base_url}}/admin/exams
Auth token needed: Yes (admin)
Body: none

[11] ADMIN RESULTS DASHBOARD

Method: GET
URL: {{base_url}}/admin/results
Auth token needed: Yes (admin)
Body: none

[12] EXAM PERFORMANCE REPORT

Method: GET
URL: {{base_url}}/reports/exam-performance
Auth token needed: Yes (admin or teacher)
Body: none

[13] PASS/FAIL REPORT

Method: GET
URL: {{base_url}}/reports/pass-fail
Auth token needed: Yes (admin or teacher)
Body: none

=====
6) RECOMMENDED DEMO ORDER (MINIMAL PAIN)
=====

A. Student flow:

1. Register student
2. Login student (auth_token auto-saved)
3. GET /users/profile
4. GET /exams
5. POST /results/submit
6. GET /results/student/{{student_id}}

B. Teacher/Admin flow:

1. Login teacher or admin
2. POST /exams

3. GET /exams
4. GET /exams/{{exam_id}}
5. GET /reports/exam-performance
6. GET /reports/pass-fail

C. Admin-only flow:

1. Login admin
2. GET /admin/exams
3. GET /admin/results

Tip:

- Keep separate environments or separate saved auth_token values per role.

7) COMMON FAILURES + FIXES

Problem: 400 Bad Request

Fix:

- Confirm body is valid JSON before encryption.
- Confirm transport_key is correct and 32-byte resolvable.
- Confirm pre-request script is attached at collection level.

Problem: 401 Unauthorized

Fix:

- Login again and verify auth_token is set.
- Confirm Authorization header is Bearer token.

Problem: 403 Forbidden

Fix:

- Wrong role for endpoint (example: student calling /admin/exams).

Problem: 422 Validation error

Fix:

- For /exams, check marks total, question format, dates, required fields.
- For /results/submit, answers array must include questionId values.

Problem: "Encrypted payload could not be decrypted"

Fix:

- transport_key does not match backend APP_ENCRYPTION_KEY.

8) OPTIONAL BUT USEFUL EXTRA ENDPOINTS (NOT IN REQUIRED 13)

- DELETE {{base_url}}/auth/logout
- GET {{base_url}}/health
- GET {{base_url}}/docs/verify (admin)

9) PROJECT FILES USED TO BUILD THIS GUIDE

- frontend/src/app/services/endpoint-docs.ts
- api/index.php
- backend/src/Http/Request.php
- backend/src/Http/AesGcmPayloadEnvelope.php
- backend/src/Services/Support/ExamPayloadValidator.php
- backend/src/Services/ResultService.php
- backend/src/Routing/Routes/*.php

END OF GUIDE